

- COMPILADO DE DECISIONES DE ARQUITECTURA - EASYDIESEL
- [ADR-001] Estilo Arquitectónico en Capas
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-002] Estilo Cliente-Servidor
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-003] Uso de TypeScript en todo el Stack
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-004] Motor PostgreSQL y ORM Prisma
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-005] Estrategia de Autenticación JWT
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-006] Comunicación mediante API RESTful
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-007] Gestión de Estado con Context API
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-008] Estrategia de Auditoría Transversal
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-009] Modelo de Concurrencia Node.js
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE
- [ADR-010] Estrategia de Despliegue Cloud
 - SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

COMPILADO DE DECISIONES DE ARQUITECTURA - EASYDIESEL

[ADR-001] Estilo Arquitectónico en Capas

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo organizar el código fuente de EasyDiesel para asegurar que sea mantenible, escalable y que la lógica de negocio no dependa de la tecnología externa (Bases de datos, Frameworks).

Orientación de la decisión Se adopta el **Estilo Arquitectónico en Capas** (N-Tier). Se dividen las responsabilidades en: Presentación (React), Aplicación (API REST), Dominio (Servicios de Negocio) e Infraestructura (Persistencia con Prisma).

Supuestos

- Se asume que la lógica de negocio cambiará con menos frecuencia que la interfaz de usuario.
- El uso de TypeScript facilitará la comunicación entre estas capas mediante interfaces.

Restricciones

- Debe mantenerse la compatibilidad con el entorno de ejecución Node.js.

Requerimientos relacionados

- RF-Gestión de inventario.
- RF-Control de transacciones.

Alternativas consideradas

- **Alternativa 1:** Monolito sin capas (todo en controladores).
- **Alternativa 2:** Arquitectura de microservicios (demasiado compleja para la fase inicial).
- **Alternativa 3:** Arquitectura en capas (Seleccionada).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Desarrollo rápido. Desventaja: Difícil de testear y mantener.
- **Alternativa 2:** Ventaja: Escalabilidad extrema. Desventaja: Costo de infraestructura y latencia de red.
- **Alternativa 3:** Ventaja: Equilibrio perfecto entre orden y velocidad de desarrollo.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 3: Capas**.

Justificación: Permite una separación clara de responsabilidades. Si mañana se decide cambiar la base de datos PostgreSQL por otra, solo se vería afectada la capa de Infraestructura, manteniendo el resto del sistema intacto.

Decisiones relacionadas

- ADR-002: Estilo Cliente-Servidor.
- ADR-004: Uso de Prisma ORM.

Comentarios Esta estructura facilita el onboarding de nuevos desarrolladores al tener carpetas bien definidas por responsabilidad.

[ADR-002] Estilo Cliente-Servidor

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo distribuir el procesamiento y la interfaz de usuario para permitir que múltiples estaciones de servicio accedan a la información de forma centralizada.

Orientación de la decisión Se adopta el estilo **Cliente-Servidor**. El cliente (Frontend React) se encarga de la interacción con el usuario y el servidor (Backend Node.js) de la lógica de datos y seguridad.

Supuestos

- La mayoría del procesamiento pesado se realizará en el servidor.
- La comunicación se realizará a través de redes locales o internet.

Restricciones

- El servidor debe ser capaz de manejar múltiples conexiones concurrentes sin pérdida de datos.

Requerimientos relacionados

- Acceso remoto desde diferentes dispositivos.
- Centralización de la base de datos de precios.

Alternativas consideradas

- **Alternativa 1:** Aplicación de escritorio con base de datos local (Peer-to-Peer).
- **Alternativa 2:** Aplicación Web Cliente-Servidor (Seleccionada).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Funciona sin internet. Desventaja: Imposible centralizar precios y auditoría.

- **Alternativa 2:** Ventaja: Datos siempre sincronizados y auditoría en tiempo real.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2: Cliente-Servidor**. Justificación: Es la forma más eficiente de asegurar que todas las estaciones de servicio operen con los mismos precios vigentes y que la administración central pueda ver los niveles de inventario al instante.

Decisiones relacionadas

- ADR-006: Comunicación mediante API REST.

Comentarios El servidor se diseña para ser "Stateless" para facilitar el escalado horizontal.

[ADR-003] Uso de TypeScript en todo el Stack

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo reducir los errores de programación relacionados con tipos de datos incorrectos en un sistema que maneja transacciones financieras y de inventario.

Orientación de la decisión Adopción de **TypeScript** como lenguaje principal tanto en el Frontend como en el Backend del proyecto.

Supuestos

- El equipo tiene conocimientos en JavaScript/TypeScript.
- El uso de tipos mejorará la autocompletación y la documentación del código.

Restricciones

- Requiere un paso de compilación adicional (transpiling) antes de la ejecución.

Requerimientos relacionados

- Integridad de los datos transaccionales.
- Documentación técnica del código fuente.

Alternativas consideradas

- **Alternativa 1:** JavaScript puro (ES6+).
- **Alternativa 2:** TypeScript (Seleccionada).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Mayor rapidez de desarrollo inicial. Desventaja: Errores en tiempo de ejecución difíciles de detectar.

- **Alternativa 2:** Ventaja: Detección de errores en tiempo de compilación y mejor mantenibilidad.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2:**

TypeScript. Justificación: La seguridad que aporta TypeScript es fundamental para un sistema donde un error de tipo (ej. tratar un precio como string) podría causar pérdidas financieras o descuadres en el inventario de combustible.

Decisiones relacionadas

- ADR-004: Motor PostgreSQL y ORM Prisma (que aprovecha TS).

Comentarios Se configurará un nivel de "strict" alto en el `tsconfig.json` para maximizar los beneficios.

[ADR-004] Motor PostgreSQL y ORM Prisma

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo gestionar la persistencia de datos de forma segura, relacional y eficiente, minimizando el código SQL manual propenso a errores.

Orientación de la decisión Uso de **PostgreSQL** como motor de base de datos relacional y **Prisma ORM** como herramienta de mapeo objeto-relacional.

Supuestos

- Los datos del sistema son altamente relacionales (Usuarios -> Roles -> Estaciones -> Tanques).
- Se requiere soporte para transacciones atómicas.

Restricciones

- Dependencia de la sintaxis y limitaciones del ORM Prisma.

Requerimientos relacionados

- Consistencia de datos.
- Generación de reportes complejos.

Alternativas consideradas

- **Alternativa 1:** MongoDB (NoSQL).
- **Alternativa 2:** PostgreSQL con SQL plano (pg-node).
- **Alternativa 3:** PostgreSQL con Prisma ORM (Seleccionada).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Flexibilidad de esquema. Desventaja: Falta de integridad referencial estricta.
- **Alternativa 2:** Ventaja: Control total del SQL. Desventaja: Desarrollo lento y difícil de mantener.
- **Alternativa 3:** Ventaja: Tipado automático, migraciones fáciles y alta productividad.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 3:**

PostgreSQL + Prisma. Justificación: PostgreSQL ofrece la robustez necesaria para transacciones de combustible, y Prisma garantiza que el acceso a estos datos desde el código TypeScript sea seguro y coherente con el esquema definido.

Decisiones relacionadas

- ADR-008: Registro de Auditoría (implementado mediante disparadores o lógica de Prisma).

Comentarios Se utilizará Prisma Migrate para gestionar los cambios en el esquema de la base de datos de forma controlada.

[ADR-005] Estrategia de Autenticación JWT

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo gestionar el acceso seguro a la plataforma para diferentes tipos de usuarios (Administradores, Estaciones, Distribuidores) de forma escalable.

Orientación de la decisión Implementación de autenticación basada en **JWT (JSON Web Tokens)** y soporte multi-proveedor (Local + Google OAuth).

Supuestos

- Los usuarios prefieren opciones de inicio de sesión rápido (Google).
- El sistema debe ser capaz de identificar al usuario en cada petición sin consultar la base de datos constantemente.

Restricciones

- Necesidad de gestionar de forma segura las "Secret Keys" para la firma de los tokens.

Requerimientos relacionados

- Seguridad de la información.
- Gestión de perfiles de usuario.

Alternativas consideradas

- **Alternativa 1:** Sesiones basadas en Cookies/Server-side.
- **Alternativa 2:** JWT (Stateless) (Seleccionada).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Fácil de invalidar sesiones. Desventaja: Difícil de escalar horizontalmente.
- **Alternativa 2:** Ventaja: Escalabilidad total y compatibilidad con múltiples clientes (Web, Móvil). Desventaja: Complejidad en la invalidación de tokens.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2: JWT**.

Justificación: Permite que el servidor sea más eficiente al no tener que almacenar sesiones en memoria, y facilita la integración futura con otros servicios o aplicaciones móviles de EasyDiesel.

Decisiones relacionadas

- ADR-002: Estilo Cliente-Servidor.

Comentarios Se utilizarán Refresh Tokens para mejorar la experiencia de usuario y la seguridad.

[ADR-006] Comunicación mediante API RESTful

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo exponer las funcionalidades del backend al frontend y a terceros de forma estándar y comprensible.

Orientación de la decisión Diseño y desarrollo de una **API RESTful** utilizando el estándar HTTP y formato de datos JSON.

Supuestos

- HTTP es el protocolo universal de comunicación web.
- JSON es ligero y fácil de procesar por el frontend de React.

Restricciones

- Debe seguir las convenciones de nombres y verbos HTTP (GET, POST, PUT, DELETE).

Requerimientos relacionados

- Interoperabilidad entre sistemas.
- Consumo de servicios desde el frontend.

Alternativas consideradas

- **Alternativa 1:** GraphQL.
- **Alternativa 2:** API RESTful (Seleccionada).
- **Alternativa 3:** WebSockets (solo para tiempo real).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Flexibilidad en las consultas. Desventaja: Curva de aprendizaje y complejidad innecesaria para este proyecto.
- **Alternativa 2:** Ventaja: Simplicidad, cacheable y estándar mundial.
- **Alternativa 3:** Ventaja: Tiempo real. Desventaja: No es apto para todas las operaciones CRUD.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2: API REST**.

Justificación: Es la opción más madura y con mejores herramientas de prueba (Postman, Swagger). Permite una integración limpia con React y es suficiente para todos los requerimientos de EasyDiesel.

Decisiones relacionadas

- ADR-001: Arquitectura en Capas.

Comentarios Se utilizarán códigos de estado HTTP estándar (200, 201, 400, 401, 500) para informar errores.

[ADR-007] Gestión de Estado con Context API

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo manejar los datos globales de la aplicación (usuario autenticado, estación actual, tema) en el frontend sin complicar la estructura de componentes.

Orientación de la decisión Uso de **React Context API** junto con **Custom Hooks** para la gestión del estado global.

Supuestos

- El estado global no es extremadamente complejo (no requiere Redux).
- Se prioriza la legibilidad del código y la facilidad de mantenimiento.

Restricciones

- Evitar re-renders innecesarios en componentes grandes.

Requerimientos relacionados

- Experiencia de usuario fluida (SPA).
- Persistencia de sesión en el navegador.

Alternativas consideradas

- **Alternativa 1:** Redux / Toolkit.
- **Alternativa 2:** Context API (Seleccionada).
- **Alternativa 3:** Pasar datos por props (Prop Drilling).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Excelente para estados masivos. Desventaja: Mucho código "boilerplate".
- **Alternativa 2:** Ventaja: Nativo de React, ligero y suficiente para la escala de EasyDiesel.
- **Alternativa 3:** Ventaja: Sin librerías extra. Desventaja: Código sucio y difícil de seguir.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2: Context API**. Justificación: Para EasyDiesel, Context API proporciona el equilibrio perfecto entre potencia y simplicidad, permitiendo que componentes como el Navbar o los Dashboards accedan a la info del usuario sin complicaciones.

Decisiones relacionadas

- ADR-005: Autenticacion (El token se guarda en el contexto).

Comentarios Se utilizarán hooks especializados como `useAuth` o `useInventario` para encapsular la lógica.

[ADR-008] Estrategia de Auditoría Transversal

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo asegurar que cada cambio sensible en el sistema (ventas, cambios de precio, ajustes de inventario) sea rastreable y auditable por entes de control.

Orientación de la decisión Implementación de un sistema de **Auditoría Transversal** mediante una tabla centralizada vinculada a todas las operaciones de negocio.

Supuestos

- Cualquier error en el sistema debe poder ser investigado mirando los registros históricos.
- Los registros de auditoría son inmutables (no se borran).

Restricciones

- El registro de auditoría no debe penalizar significativamente el rendimiento de la aplicación.

Requerimientos relacionados

- Cumplimiento normativo (Sicom, DIAN).
- Seguridad y trazabilidad.

Alternativas consideradas

- **Alternativa 1:** Logs en archivos de texto.
- **Alternativa 2:** Tabla de Auditoría en Base de Datos (Seleccionada).
- **Alternativa 3:** Disparadores (Triggers) de base de datos.

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Muy rápido. Desventaja: Difícil de consultar y cruzar con datos de usuario.
- **Alternativa 2:** Ventaja: Fácil de reportar, cruzar con entidades y asegurar mediante transacciones.
- **Alternativa 3:** Ventaja: Automático. Desventaja: Difícil de mantener y limita la lógica de aplicación.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2: Tabla de Auditoría**. Justificación: Permite generar reportes de auditoría directamente desde la aplicación para los administradores, asegurando que cada acción tenga un responsable y un contexto claro.

Decisiones relacionadas

- ADR-004: Base de Datos.

Comentarios Se almacenarán los datos "Antes" y "Después" de cada operación en formato JSON.

[ADR-009] Modelo de Concurrencia Node.js

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo manejar la concurrencia de múltiples usuarios (varios isleros vendiendo al tiempo) sin bloquear los recursos del servidor.

Orientación de la decisión Aprovechamiento del modelo de **E/S No Bloqueante y Event Loop** nativo de Node.js.

Supuestos

- La mayoría de las operaciones son de entrada/salida (esperar a la base de datos).
- Node.js gestiona eficientemente miles de conexiones simultáneas en un solo hilo.

Restricciones

- Evitar tareas que bloqueen el hilo principal (cálculos matemáticos masivos síncronos).

Requerimientos relacionados

- Alta disponibilidad.
- Rendimiento bajo carga.

Alternativas consideradas

- **Alternativa 1:** Multi-threading (Java/C#).
- **Alternativa 2:** Event-Driven (Node.js) (Seleccionada).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Mejor para uso intensivo de CPU. Desventaja: Mayor consumo de memoria por cada hilo.
- **Alternativa 2:** Ventaja: Extremadamente eficiente en memoria y escalable para aplicaciones de red.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2: Node.js Event Loop**. Justificación: EasyDiesel es una aplicación intensiva en I/O (comunicación con DB y Clientes). Node.js es ideal para este perfil de carga, permitiendo respuestas rápidas incluso con muchos usuarios concurrentes.

Decisiones relacionadas

- ADR-002: Estilo Cliente-Servidor.

Comentarios Se utilizará `async/await` para mantener el código limpio y manejar la asincronía.

[ADR-010] Estrategia de Despliegue Cloud

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/Preocupación de diseño Cómo garantizar que la aplicación pueda ser desplegada y escalada en diferentes entornos (Desarrollo, Pruebas, Producción) sin cambios en el código.

Orientación de la decisión Estrategia de **Externalización de Configuración** mediante variables de entorno (`.env`) y preparación para despliegue Cloud.

Supuestos

- Las credenciales de base de datos y llaves de API cambian según el entorno.
- Se busca evitar la exposición de secretos en el repositorio de código.

Restricciones

- Necesidad de un sistema de gestión de secretos seguro en producción.

Requerimientos relacionados

- Despliegue continuo (CI/CD).
- Seguridad de infraestructura.

Alternativas consideradas

- **Alternativa 1:** Archivos de configuración hard-coded.
- **Alternativa 2:** Variables de entorno (Seleccionada).

Evaluación de alternativas

- **Alternativa 1:** Ventaja: Ninguna. Desventaja: Inseguro y rígido.

- **Alternativa 2:** Ventaja: Sigue los principios de "12-Factor App", seguro y flexible para la nube.

Alternativa seleccionada y Justificación Se selecciona la **Alternativa 2: Variables de Entorno**. Justificación: Permite que el mismo artefacto de software corra en cualquier servidor. Es la base para usar servicios Cloud como AWS, Render o Vercel, facilitando el despliegue de EasyDiesel.

Decisiones relacionadas

- ADR-004: Base de Datos (URL de conexión externa).

Comentarios Se utiliza la librería `dotenv` en desarrollo para cargar las variables locales.